CIS 341 Fall 2026 course project

20% of Course grade

10/22/26

System Health Monitor & Security Auditing Daemon

Objective: Design and Implement a Linux System Health Monitor & Security Auditing Daemon

Project Framework: You can leverage Shell script (Bash) or Python for the project

Functional Specifications (100%)

1. Detailed Interactive USAGE Engine (2.5%)

Whenever the program is run manually from the command line without args or with a help flag (e.g., -h or --help), it must immediately intercept execution and dump an exhaustive Program USAGE Statement to standard output. This statement must document:

Detailed instructions and operational syntax configurations with copy-pasteable execution examples.

Exact sample results of output logs, metrics strings, and console warnings.

Every parameter option consumed by the CLI engine.

The internal dictionary schema mapping and format layout of its configuration file.

A comprehensive error code dictionary mapping failure index to human-readable exit descriptions.

2. Spontaneous Initialization Service Setup (2.5%)

The program must be built to run natively as a detached background system engine or daemon. It must integrate cleanly with the Linux initialization architecture by including a customized systemd service unit file (e.g., sysaudit.service) that automatically launches the daemon in background multi-user processing modes when the Linux operating system boots up.

3. Chronological Cron/Timer Execution (7.5%)

The daemon must run an internal tracking loop or be triggered by an external scheduler (like systemd timers or cron) at midnight every 3 days to perform maintenance cleanups inside its workspace folder located at /course_project/audit/.

It must discover all ancient raw report strings (*.audit) generated by the scanner.

It must safely compress and package them into an archive format (e.g., .zip or .tar.gz).

It must completely delete the individual processed raw text source files immediately after a successful compression workflow to save disk spaces.

4. Dynamic Decoupled Configuration Architecture (5.0%)

All static options, ceiling indices, target folders, and default behaviors must be mapped outside the program code space. The engine must natively read and parse its own external custom configuration layout file named sysaudit.cfg. Hardcoded runtime settings inside the main script logic are strictly forbidden.

5. Isolated Self-Monitoring Transcript Engine (7.5%)

The program must implement a dedicated internal auditing path. It must generate and update its own structural trace log (e.g., /var/log/sysaudit_daemon.log) to record its own execution health. Every scan loop initialization, successful data storage file dump, script sleep cycle configuration update, and handled exception parameter event must be strictly tracked with a clear severity level layout ([INFO], [WARN], [ERROR]) and standardized timestamps.

6. Automated Circular Storage Pruning (5.0%)

To maintain a safe local hard drive envelope, the script must calculate data lifetimes during its cleanup phase. It must locate and permanently delete any archived compression volumes that are older than 2 weeks.

7. Core Metadata Audit Summary Logging (5.0%)

During each archive loop execution, the program must calculate structural file statistics and write a summary entry into its log. This summary must record:

The absolute total number of current compressed archives sitting inside the repository folder.

The exact string name of the largest compressed archive file discovered in the directory.

The precise byte calculation size of that largest file.

The high-fidelity metadata file modification timestamp recording exactly when that file creation occurred.

8. Configurable Resource Ceiling Warning Interrupts (5.0%)

The program must actively evaluate disk storage consumption metrics. If the total aggregated size of all elements trapped within the /course_project/audit/ target tree hits an over-allocation baseline limit (e.g., 100MB), the script must instantly trigger a high-priority [WARNING] notice flag inside its transaction file. This maximum safety constraint envelope value must be completely configurable within the sysaudit.cfg properties layout.

9. Upstream Parameter Precedence Mapping (5.0%)

The programming logic must evaluate arguments dynamically based on clear precedence rules. While the application initializes options natively from the sysaudit.cfg text mapping, passing arguments manually via the CLI execution path must immediately overwrite those baseline parameters for that specific process invocation loop instance.

10. Robust Exception Trapping & Exit Architecture (5.0%)

The program logic must avoid unhandled interpreter fatal crashes at all costs. All system interaction points, directory lookups, file operations, hardware checks, and string manipulation logic blocks must be safely wrapped inside granular exception validation loops. If a critical runtime fault occurs, the program must trap the issue, gracefully document the failure reason to the diagnostic standard error stream (STDERR), and terminate execution by returning an explicit, unique non-zero exit status code.

11. Locked User Execution Space Sandboxing (5.0%)

The script must actively query system session details upon execution. It must verify the effective execution user account name running the process loop and enforce a security policy that allows the program to execute *only* if the session user string matches auditmanager exactly. If any other user tries to call the application, it must reject access immediately and terminate with a security privilege violation code.

12. Secure Role Delegation & Autonomy Allocation (5.0%)

To simulate production workflow handoffs, the script must provide an explicit functional argument parameter loop that allows the primary user auditmanager to safely delegate execution validation rights or file domain group access to an alternate specified local Linux operating system user identifier.

13. Comprehensive Functionality Verification Suites (15.0%)

Students must design and implement a structured validation testing layout containing sufficient test coverage cases. This testing script array must automatically execute and simulate:

Standard boundary configurations, missing files, corrupted target paths, and malformed configuration configurations.

Over-allocation storage limits to confirm warning flag triggers.

Unprivileged account validation hacks to confirm sandboxing barriers.

14. Architecture Technical Presentation Profile (25.0%)

Students must build a complete engineering presentation document mapping out:

Complete structural bottleneck analysis and operational optimization constraints.

System design maps showing data loops, pipeline transfers, and file state flows.

High-fidelity implementation proofs demonstrating deployment execution across test environments.

Implementation Guide & Infrastructure Skeleton

A. Recommended sysaudit.cfg Configuration Format Layout

You can format the external options file using a clean key-value syntax model (like an .ini layout or a clean JSON file). It must document clear variable paths without spaces around the assignment operators where possible.

B. Suggested System Error Code Dictionary Structure

To guarantee robust exception handling, you are encouraged to create an internal lookup index mapping specific exceptions to the POSIX-aligned exit codes:

Hex / Dec Code	Logical Identifier	Functional Failure Meaning
0	SUCCESS	Operational task ran to completion without tracking loops faults.
1	GENERIC_ERROR	An unmapped error condition triggered a process escape.
2	IDENTITY_VIOLATION	Process call called by an unverified account other than auditmanager.
3	CONFIGURATION_MISSING	The explicit target configuration layout file sysaudit.cfg was not found.
4	INVALID_WORKSPACE	Target mapping path /course_project/audit missing or is not a directory.
5	DISK_CEILING_REACHED	Workspace capacity validation failed critical storage envelope margins.